

Underwater Flappy — Architecture & Build Plan v2.0

Web-First, Mobile-Ready · 4 Phases · Agentic Team Execution

Audience: Claude Code / CLI agents executing this project end-to-end. **Supersedes:** Unity 2022 LTS plan v1.0. Unity is retained only as a contingency (see §9).

1. Product Definition (unchanged from v1)

A Flappy Bird-style 2D game set underwater. The **core differentiator is fluid dynamics**: buoyancy, water drag, and momentum replace flappy's dry gravity-snap feel. Classic mode keeps flappy's simplicity — one input (tap), procedural obstacles, one-hit death, score-per-gate.

- **Classic mode (this plan):** low-fidelity stylized fluid physics — buoyancy + drag + momentum. Simple, addictive, 60fps everywhere.
 - **Normal/Advanced modes (future):** currents, vortices, multiple creatures, particle turbulence. Architecture must leave seams for these (see FluidField interface, §4.3) but **MUST NOT** implement them now.
 - **Platforms:** Web (browser) first — this is the build/iteration arena. iOS + Android via Capacitor in Phase 4. Same codebase, no rewrite.
-

2. Stack Decision

Layer	Choice	Why
Language	TypeScript (strict)	Type safety for agentic dev; refactor-friendly
Engine	Phaser 3 (latest 3.x)	Mature 2D WebGL engine, scene system, input, audio, tweens
Physics	Phaser Arcade + custom FluidBody layer	Arcade is the fastest option; our fluid model is bespoke math (buoyancy/drag) applied as forces — keeping it in our own module makes it portable and tunable. Do NOT use Matter.js (overkill, slower on mobile).
Build	Vite	Instant HMR for the iteration loop; tiny prod bundles

Layer	Choice	Why
Tests	Vitest (unit/sim), Playwright (e2e/smoke)	Physics is pure functions → unit-testable without a browser
Mobile wrapper	Capacitor 6+	WebView wrapper; one codebase to App Store + Play Store
State persistence	localStorage (web) → Capacitor Preferences (mobile) behind a Storage interface	
CI	GitHub Actions: lint → typecheck → unit → build → e2e	Every phase gate runs in CI
Assets	Kenney.nl / OpenGameArt placeholders → custom art later	Per v1 plan; avoids licensing issues

Why web-first beats Unity for this project: sub-second iteration loop (Vite HMR vs Unity domain reload), instantly shareable builds (a URL is a playtest), physics is ~300 lines of pure TS we fully own, and Capacitor closes the mobile gap for a casual 2D game. Unity's advantages (3D, heavy simulation, console) don't apply here.

3. Repository Layout

```

underwater-flappy/
├─ ARCHITECTURE.md          # this file - agents read it first
├─ docs/
│   ├─ PHYSICS_SPEC.md     # formulas, constants, tuning ranges
│   ├─ TEST_PLAN.md        # per-phase test matrix
│   └─ DECISIONS.md        # ADR log - every agent decision appended here
├─ src/
│   ├─ main.ts              # bootstrap, Phaser config
│   ├─ config/
│   │   └─ physics.json     # ALL tunable constants (single source of
truth)
│   │   └─ difficulty.json  # spawn rates, gap sizes, speed curves
│   └─ core/
│       └─ fluid/

```

```

|   |   |   |— FluidBody.ts    # buoyancy + drag force model (pure functions)
|   |   |   |— FluidField.ts  # interface: sampleForce(x,y) – Classic
returns constant; future modes return currents
|   |   |   |— index.ts
|   |   |   |— spawn/          # obstacle generator + object pool
|   |   |   |— score/          # scoring, combo, persistence
|   |   |   |— state/          # finite state machine:
BOOT>MENU>PLAY>DEAD>MENU
|   |— scenes/                 # Phaser scenes: Boot, Menu, Game, GameOver,
HUD
|   |— entities/               # Player, Obstacle, Bubbles (visual only)
|   |— platform/
|   |— Storage.ts              # interface + web impl; Capacitor impl in
Phase 4
|   |— Haptics.ts              # no-op on web; native in Phase 4
|   |— ui/
|— tests/
|   |— unit/                   # fluid math, spawn logic, scoring, FSM
|   |— sim/                    # headless gameplay simulations (determinism,
fairness)
|   |— e2e/                    # Playwright: boot, play, die, restart,
persist score
|— capacitor/                  # created in Phase 4 only
|— .github/workflows/ci.yml

```

Architectural rules (binding for all agents):

1. `core/` has **zero Phaser imports**. Pure TS, deterministic, unit-tested. Phaser scenes consume core via thin adapters. This is what makes the code portable (to Capacitor, or worst-case to any other engine).
 2. All physics/difficulty constants live in `config/*.json`. No magic numbers in code. Tuning = editing JSON, hot-reloaded in dev.
 3. Fixed timestep simulation (1/60s accumulator) inside core; rendering interpolates. Identical inputs $\Rightarrow$ identical run (seeded RNG for obstacle spawning). This makes gameplay replayable and testable.
 4. Every public module gets unit tests before the phase gate closes.
-

4. Fluid Physics Spec (Classic)

4.1 Forces per fixed step (pure function in `FluidBody.ts`)

```
F_gravity = m · g // g tuned low, e.g. 600 px/s2
(water feel)
F_buoyancy = -p · V · g · k_b // constant upward; net sink slight
F_drag = -c_d · v · |v| // quadratic drag – THE underwater
feel
F_swim = impulse on tap, applied as velocity change with easing (not
instant set)
v_max = clamped terminal velocities (rise/sink asymmetric)
```

Tap does **not** zero velocity like flappy; it adds an upward impulse blended over 2–3 frames, so momentum carries through. Rotation of sprite follows velocity vector with damped lerp.

4.2 Feel targets (acceptance criteria, Phase 1)

- Idle player sinks slowly (~80–120 px/s terminal).
- Single tap arrests sink and produces gentle rise arc; double-tap chains feel "swimmy," not "rockety."
- No frame of motion looks instantaneous; everything eases. Side-by-side video vs. original Flappy Bird must read as obviously "underwater."

4.3 Future-proofing seam

`FluidField.sampleForce(x, y, t)` — Classic mode implementation returns a constant `(0, buoyancyNet)`. Normal mode later swaps in a vector-field implementation (currents/vortices) **without touching FluidBody or Player**. Document this in `PHYSICS_SPEC.md`; do not build the vector field now.

5. Agentic Team Structure

One orchestrator session drives subagents. Roles map to Claude Code subagents/tasks; each role has a contract.

Agent	Owns	Definition of Done
Orchestrator/Architect	Phase sequencing, ADRs in DECISIONS.md, merge approval	Phase gate checklist green; ADR written for any deviation from this doc
Physics Agent	<code>core/fluid/</code> , physics.json, PHYSICS_SPEC.md	Unit tests pass; sim tests confirm feel targets numerically (terminal velocities, arc heights within spec $\pm 5\%$)
Gameplay Agent	<code>core/spawn</code> , <code>core/score</code> , <code>core/state</code> , entities	Deterministic sim tests pass; fairness test (every spawned gap is passable given physics constraints)
UI/Scene Agent	<code>scenes/</code> , <code>ui/</code> , HUD, menus, juice (tweens, particles, screen shake)	Playwright e2e flows pass; 60fps in Chrome DevTools perf trace
QA Agent	<code>tests/</code> ownership, TEST_PLAN.md, regression suite, perf budgets	Writes tests BEFORE features merge (gameplay agent implements against them); maintains coverage $\geq 80\%$ on core/
DevOps Agent	Vite config, CI, deploy (GitHub Pages/Netlify), Phase-4 Capacitor builds	CI green on main; preview URL on every PR; signed store builds in Phase 4

Working agreement: QA Agent writes acceptance tests from this doc's phase gates first → feature agents implement to green → Orchestrator merges. Every merge to main must deploy a playable preview URL. Humans playtest from the URL and feed tuning notes back as physics.json diffs.

6. The Four Phases

Phase 1 — Foundation & Physics Sandbox (Days 1-4)

Goal: the fluid feel exists and is provably right. This phase is the whole game's risk.

Tasks:

1. DevOps: repo scaffold, Vite+TS+Phaser, Vitest, Playwright, CI pipeline, auto-deploy previews.
2. Physics: implement FluidBody + FluidField (constant), physics.json, fixed-timestep core loop with seeded RNG.
3. Gameplay: minimal FSM (BOOT→PLAY only), player entity bound to fluid core.
4. UI: sandbox scene — player in empty water column, on-screen debug sliders bound to physics.json values, FPS counter.
5. QA: unit tests for all force functions; sim test harness that runs N seconds of headless simulation and asserts terminal velocity, tap arc height/duration, energy decay.

Gate G1: sandbox playable at preview URL; feel targets (§4.2) verified by sim tests AND human playtest sign-off; CI green; 60fps on a mid-range Android phone in Chrome.

Phase 2 — Classic Mode Complete (Days 5-10)

Goal: full game loop — menu, play, die, score, restart.

Tasks:

1. Gameplay: obstacle spawner (object-pooled, seeded), collision (Arcade overlap vs. tight hitboxes ~80% of sprite), scoring per gate, difficulty curve from difficulty.json (speed/gap tighten over score).
2. Gameplay: full FSM — Menu → Play → Dead (death sequence: drift + slow-mo 300ms) → GameOver → restart in <1s.
3. UI: Menu scene, HUD (score, best), GameOver panel, tap-to-start; placeholder art pass (Kenney assets), parallax background layers, bubble particle trail.
4. Core: Storage interface + localStorage impl; persist best score.
5. QA: fairness sim (10,000 seeded runs: every gap reachable from worst-case entry velocity); collision precision tests; e2e: full play-die-restart-persist loop; perf budget test (draw calls, heap stable over 10-min soak).

Gate G2: complete game playable start-to-finish at URL; fairness suite green; no GC stutter in 10-min soak; restart <1s; best score survives reload.

Phase 3 — Polish, Audio, Web Release (Days 11-15)

Goal: ship-quality web release. This is the public "building arena" — live URL gathering feedback while mobile work proceeds.

Tasks:

1. UI: juice pass — squash/stretch on tap, screen shake on death, score pop, caustic light shader (cheap), water-tint vignette. All effects toggleable for perf.

2. Audio: Web Audio via Phaser — muffled underwater SFX (tap, score, death), loopable ambient; mute toggle persisted.
3. Gameplay: pause/resume on visibility change; resize/orientation handling (portrait lock layout, letterbox).
4. DevOps: production build budget — initial load <3MB, time-to-playable <3s on 4G; PWA manifest + service worker (installable, offline-capable); deploy to production domain; basic analytics (privacy-light: session, score distribution).
5. QA: cross-browser matrix (Chrome/Safari/Firefox, iOS Safari, Android Chrome); Lighthouse ≥90 performance; accessibility pass (reduced-motion respect, contrast); regression suite locked as the Phase-4 safety net.

Gate G3: public URL live; Lighthouse ≥90; works on iOS Safari + Android Chrome at 60fps (30fps floor on low-end with effects auto-reduced); zero P0/P1 bugs open.

Phase 4 — Mobile Packaging & Store Release (Days 16–22)

Goal: same codebase on the App Store and Play Store.

Tasks:

1. DevOps: add Capacitor; iOS (Xcode, ARM64, signing per Apple Developer account) and Android (AAB, target latest API) projects; CI lanes for both builds.
2. Platform: swap Storage → Capacitor Preferences; add Haptics (light tap on score, heavy on death); safe-area insets (notch); keep-awake during play; status bar hidden.
3. QA: **device benchmark gate** — 60fps sustained on iPhone SE-class and a 2-3-year-old mid-range Android (e.g., Pixel 6a-class). Input latency tap-to-impulse <50ms. Battery: <15%/30min. **If this gate fails after one optimization sprint, trigger Unity contingency (§9).**
4. Gameplay: monetization deferred to post-launch v1.1 (rewarded continue + remove-ads IAP) — but stub the Ads/IAP platform interfaces now so v1.1 is additive. (ADR: launching clean improves store review odds and keeps Phase 4 short.)
5. DevOps: store assets (icons, screenshots, preview video from web build), privacy policy, content ratings, submit. Play review ~1–3 days, iOS ~1–7 days.

Gate G4: both store submissions accepted; release builds pass full regression suite; crash-free 1-hour soak on both reference devices.

7. Testing Strategy Summary (QA Agent's charter)

Layer	Tool	What	When
Unit	Vitest	Fluid math, spawn logic, scoring, FSM transitions	Every commit
Simulation	Vitest (headless core)	Determinism, feel-target numbers, fairness (10k seeded runs), difficulty curve sanity	Every commit
E2E	Playwright	Boot→play→die→restart→persist; pause/resume; mute persist	Every PR
Performance	Playwright + CDP traces	FPS, heap soak, load budget, Lighthouse	Phase gates + nightly
Device	Manual + Capacitor builds	Reference-device benchmarks, haptics, safe areas	Phase 4 gate

Coverage floor: 80% on core/. Scenes/UI covered by e2e, not unit tests.

8. Risks & Mitigations

Risk	Mitigation
WebView perf on old Android	Effects quality tiers auto-selected by FPS probe at boot; Arcade physics (cheapest); pooling everywhere
iOS Safari audio unlock quirks	Audio init gated on first user gesture (standard pattern); tested in G3 matrix
Physics feels wrong subjectively	Phase 1 is dedicated to feel with live-tunable JSON + instant preview URLs; nothing else builds until G1 signs off
Scope creep into Normal-mode features	FluidField seam documented; orchestrator rejects any vector-field/current PR until post-launch
Capacitor gate failure	§9 contingency, decided by benchmark data, not opinion

9. Unity Contingency (escape hatch only)

Trigger: Phase 4 device benchmark gate fails after one focused optimization sprint. Because `core/` is pure TS with documented formulas (PHYSICS_SPEC.md) and all tuning in JSON, a Unity port re-implements ~600 lines of deterministic logic against the same sim test expectations — the design, tuning values, fairness data, art, and store assets all carry over. Estimated port: 2–3 weeks per the v1.0 Unity guide, which remains in the project archive. This is insurance; for this genre, the gate is expected to pass.

10. Post-Launch Roadmap (out of scope, for context)

v1.1 monetization (rewarded continue, remove-ads IAP) → v1.2 Normal mode (FluidField vector currents, second creature) → leaderboards.

11. Kickoff Instructions for Claude Code

1. Read this file fully. Append an ADR to docs/DECISIONS.md acknowledging the plan.
2. Execute Phase 1 task list in order; spawn subagents per §5 roles.
3. Never merge to main with red CI. Never close a phase without its gate checklist
written into docs/DECISIONS.md with evidence (test output, perf trace, preview URL).
4. All tuning changes are physics.json/difficulty.json diffs with a one-line rationale.
5. When blocked or deviating from this document, write an ADR before coding.

Document version 2.0 — supersedes Unity plan v1.0 — prepared for agentic execution.